LLMKernelBench: Benchmarking Large Language Models on Software Vulnerability Detection in Linux Kernel

Dear reviewers, thank you for the constructive comments. We have revised the paper according to your recommendations.

NOTE: Upon auditing the raw model responses, we found that the original parser conflated malformed outputs with abstentions and also failed to recognize some genuine `not sure` answers. We therefore reclassified raw responses into valid binary verdicts, genuine abstentions, and invalid/indeterminate outputs. Genuine abstention is uncommon but nonzero (2.40% for GPT-4.1-mini when averaged across the two $T = 0.1$ runs), while invalid output is concentrated in StarCoder2 (up to 26.08%). We apologize for the earlier misclassification and have updated the methodology, results, appendix, and discussion accordingly.

I. Reviewer 1

Section IV-A (Non-targeted Classification): Provide a brief qualitative discussion or hypothesis on the extreme differences in model abstention behavior (e.g., Llama3.1 abstaining 60% of the time versus Mistral at 0%). Does this correlate with the reinforcement learning from human feedback (RLHF) alignment strategies of the respective models?

We thank the reviewer for the question. Upon re-examining the raw generations, we found that the originally reported Llama3.1 abstention rate of approximately 60% was an artifact: malformed or unresolved generations had been treated as if they expressed uncertainty. We now distinguish *genuine abstention*, where the model clearly selects `not sure` because the available evidence is insufficient, from *invalid output*, where the model continues code, loops, gives contradictory choices, or otherwise fails to produce a resolvable verdict. In the primary $T = 0$ audit, genuine abstention is 0% for all open-source models except Qwen3-Coder (0.06%), while StarCoder2 produces 26.08% invalid output. At $T = 0.1$, GPT-4.1-mini averages 2.40% genuine abstention across the two runs, while StarCoder2 averages 22.45% invalid output. These corrected results do not support an RLHF-based explanation for the original Llama3.1 figure. Table I gives the audited decoding summary. The revised definitions and primary-run breakdown appear in Section IV-A and Table V (page 8); the same decoding comparison appears in manuscript Table XIX (page 15).

TABLE I
Genuine-abstention/invalid-response rates (% of all responses), pooled over both SVD settings and datasets; $T = 0.1$ is averaged across two runs.

Model	T=0	T=0.1 (two-run avg.)	T=1.0
CodeLlama	0.00/1.80	0.00/1.83	2.34/1.20
DeepSeek-R1	0.00/0.00	0.00/0.00	0.00/0.00
GPT-4.1-mini	–	2.40/0.00	–
Llama3.1	0.00/0.00	0.00/0.00	0.00/0.00
Mistral	0.00/1.38	0.00/1.47	0.48/0.18
Qwen3-Coder	0.06/0.48	0.12/0.57	1.80/0.18
StarCoder2	0.00/26.08	0.00/22.45	1.14/1.92

Section IV-C (HPS Validation): The manuscript mentions "Table ??" multiple times in the "HPS Validation" paragraph (e.g., "distance $\leq$ 3, Table ??"). Ensure all table cross-references are correctly linked before the final camera-ready submission.

We thank the reviewer for pointing this out. The missing references in the HPS Validation discussion were due to tables omitted from the previous manuscript version. They have been restored and linked from Section IV-D. Specifically, the revised paper references the hierarchy-distance statistics in Table XV (page 12), the CWE-119 prediction-bias

analysis in Table XVI (page 13), the HPS-to-Top-1 gap in Table XVII (page 13), and the hierarchy-distance distribution in Table XVIII (page 13). We also added the exact distance definition and a committed script that regenerates these counts from the three ranking runs.

Section III-D (Prompt Templates): Briefly mention the exact system prompt instruction used to handle the abstention (-1) class. Was the model explicitly told "Return -1 if you are unsure"?

We thank the reviewer for the observation. The production SVD prompt explicitly instructs the model to return `not sure` when it cannot determine the class; it does not ask the model to emit the literal token `-1`. The uncertainty response is subsequently represented as -1 in the structured evaluation data. The verbatim SVD template appears in Figure 3 (page 7), and Section III-D now states this mapping explicitly.

Section V (Discussion): Add a sentence or two discussing the computational cost or inference time associated with processing Level 3 (Multiple Files) contexts, especially for models with large context windows like GPT-4.1-mini. This is highly relevant for reliability engineers considering deploying LLMs in CI/CD pipelines.

We thank the reviewer for the suggestion. We added the paragraph "Deployment cost at multi-file context" in Section V (page 12). Median Level 3 prompts are approximately 2.7× longer than Level 1 prompts. In the two released GPT-4.1-mini logs, the median sequential completion interval is 0.51 s at Level 1 and 0.53 s at Level 3 for the non-targeted PBD task; these values include API and local orchestration overhead and are not a controlled service-level benchmark. A self-hosted Llama3.1-8B projection rises from 2.5 s to 6.8 s. At the pricing snapshot used in our analysis, a typical eight-output-token GPT-4.1-mini request costs approximately $0.53 per 1,000 Level 3 predictions, with $1.34 as a conservative 512-token upper bound; the self-hosted Llama estimate is $1.61. We therefore recommend a multi-file context for targeted or diff-triggered review batches rather than indiscriminate continuous scanning.

II. Reviewer 2

-Fig 1 should be better described. E.g. Langchain is included but not even mentioned.

We thank the reviewer for pointing this out. We revised the caption of Figure 1 (page 4) and the corresponding discussion in Section III to explain that LangChain parses raw model outputs and that a separate raw-response audit distinguishes valid binary verdicts, genuine abstentions, and invalid outputs.

-Why such CWEs?

We thank the reviewer for the question. We did not pre-select a preferred subset of CWE categories. The PBD uses the CWE labels associated with the sampled Linux-kernel CVEs, while 40 LFD samples lacking NVD CWE identifiers were manually labeled under NVD-aligned criteria. Consequently, the resulting label space—74 distinct CWEs in PBD and 19 in LFD—reflects the sampled kernel vulnerabilities rather than an artificially balanced category design. Section III-A and Table III (page 5) now explain the coverage, its relationship to the CWE-1000 hierarchy, and the resulting imbalance.

-Prompt Templates should be delivered/ included in someway

We thank the reviewer for the suggestion. The manuscript includes representative templates for SVD in Figure 3 (page 7) and CWE ranking in Figure 4 (page 7). We also revised Section III-D to describe their design principles and role in the evaluation.

-Why 9 is a sensible amount of CWEs to be considered as part of the dataset? What is the reasoning behind? The dataset is limited and imbalanced and because of that, I have doubts about the strength of the results.

We thank the reviewer and wish to clarify an important distinction. The "9 CWEs" refer to nine populated top-level reporting categories in the table: eight CWE-1000 pillars plus the CWE-OTHER group, not nine individual weakness types. The dataset contains **74 distinct CWE identifiers** in PBD and **19** in LFD, as detailed in Section III-A and Table III (page 5). The categories were not selected to create an artificial balance; PBD uses the sampled CVEs' labels, and missing LFD labels were assigned according to NVD-aligned criteria. We address the resulting imbalance through the micro/macro analysis in Table XII (page 11) and hierarchy-aware HPS evaluation. Section VI further cautions that fine-grained estimates for rare CWEs are indicative rather than statistically definitive.

-Why not considering CWE detection? It is a challenging approach and most approaches deal with binary classification.

We thank the reviewer for raising this point and clarify that the paper *does* evaluate CWE identification. Models rank their five most likely CWE labels, and we report Top-1 through Top-5 accuracy, MRR, and HPS in Section IV-D (page 10). We use ranking rather than training a separate binary classifier for each CWE because the dataset spans 74 distinct weakness types, many with very few examples, making individual per-CWE classifiers unreliable and impractical to evaluate fairly. Top-k accuracy measures exact identification, while HPS separately records hierarchy-aware near misses.

-The metric concerning the proposed hierarchical structure is nice but it is difficult to interpret, e.g. in a SOC, its applicability.

We thank the reviewer for this practical observation. HPS is not proposed as a replacement for exact CWE accuracy or as a standalone SOC decision metric. It measures the taxonomic proximity of the predicted candidate set, which may still help route or prioritize manual review. We added a worked example to the CWE-label metrics discussion in Section III-C2 (page 6). A prediction of CWE-121 (Stack-based Buffer Overflow) for a true CWE-787 (Out-of-bounds Write) receives the immediate-child weight of 0.7, whereas CWE-284 (Improper Access Control) is in a different root branch and receives 0.0. The implementation preserves all parent and root-pillar memberships because CWE-1000 is a directed acyclic graph rather than a strict single-parent tree. In SOC triage, the former can still direct attention toward buffer-handling code, but it must not be treated as an exact diagnosis.

-There are many papers in which classification of C language codes/snippets gets a really high accuracy using traditional AI (e.g. using code features) and LLMs. A comparison and justification is required. For instance, in [2] it is explained "Alam et al. [2] fine-tuned GPT-4 and achieved 99% accuracy in Solidity vulnerability detection.". Another relevant paper which gets high accuracy with LLMs and uses a big dataset [1]

We thank the reviewer for highlighting these references. Their high reported accuracy (99% for fine-tuned GPT-3.5 Turbo and GPT-4o Mini in Alam et al.; 94% binary and 92% multiclass for SecureFalcon) is not directly comparable to our results because they use task-specific training data and target different domains or languages. Our study instead evaluates zero-shot, off-the-shelf models on Linux kernel C and includes a post-cutoff split. We expanded Section II (page 2) and added a direct comparison in Section IV-A1 (page 8).

-Related work should be improved, e.g. SecureFalcon

We thank the reviewer. SecureFalcon [1] is now discussed in Section II (page 2), including its fine-tuning methodology, FormAI/FalconVulnDB data, and reported performance. The revised text contrasts supervised in-distribution evaluation with our zero-shot, temporally separated setting.

-Minor: in page 13, some ?? are noticed

We thank the reviewer for pointing this out. The omitted HPS-supporting tables have been restored and referenced from Section IV-D. They are Tables XV, XVI, XVII, and XVIII.

-Concerning the sentence "Frequent Vulnerability Types Inflate Performance Metrics", has the dataset enough amount of CWEs (e.g. some of them just have 9, 11, 27) to get to this conclusion? Again, I consider the dataset limited to write some of the "findings" (specially in a high-quality journal)

We thank the reviewer for this important concern regarding validity. The revised **Frequent Vulnerability Types Are Associated with Higher Aggregate Performance** finding does not rest on any single rare-CWE estimate. It is based on the aggregate micro/macro accuracy gap across models in Table XII (page 11): 0.9 percentage points in PBD and 6.6 in LFD. This descriptive difference is consistent with frequency sensitivity, but it may also reflect the splits' different class composition and the smaller LFD sample. We therefore do not interpret it as proof of data leakage or of a particular internal model mechanism.

That said, we agree that the per-CWE estimates in Table XIII (page 11) require caution for types with fewer than approximately 20 samples. We added this caveat to Section IV-D and Section VI, clarifying that fine-grained class-level conclusions are indicative rather than statistically definitive.

References

[1] Ferrag, M. A., Battah, A., Tihanyi, N., Jain, R., Maimuţ, D., Alwahedi, F., Lestable, T., Thandi, N. S., Mechri, A., Debbah, M., et al. Securefalcon: Are we there yet in automated software vulnerability detection with llms? *IEEE Transactions on Software Engineering* (2025).

[2] Sheng, Z., Chen, Z., Gu, S., Huang, H., Gu, G., and Huang, J. Llms in software security: A survey of vulnerability detection techniques and insights. *ACM Computing Surveys 58*, 5 (2025), 1–35.